

RELAZIONE TECNICA ED OPERATIVA: STORED XSS (MEDIUM)

Analisi sequenziale delle vulnerabilità, evoluzione dei payload ed esecuzione dei dump di laboratorio

1. Introduzione e Scenario Logico

La presente relazione documenta le attività di analisi e penetration testing svolte all'interno della piattaforma didattica DVWA (Damn Vulnerable Web Application), nello specifico modulo dedicato agli attacchi Stored Cross-Site Scripting (XSS), impostato sul livello di sicurezza Medium. L'obiettivo dell'attività è illustrare in maniera analitica il superamento progressivo dei filtri di sicurezza client-side e server-side, documentando l'evoluzione dell'attacco passata attraverso due distinte fasi operative.

La vulnerabilità di tipo Stored (o memorizzata) rappresenta il livello più insidioso di XSS poichè il codice maligno, una volta iniettato tramite un modulo vulnerabile, viene archiviato in maniera persistente all'interno del database relazionale del server web. Di conseguenza, l'esecuzione del codice non richiede un'azione mirata verso una singola vittima tramite link malevoli, ma si innesca in modo del tutto automatico e trasparente nel browser di qualunque utente legittimo che navighi sulla pagina compromessa.

2. Analisi del Codice Sorgente PHP Vulnerabile

Il fallimento della sicurezza nel livello Medium risiede in una logica difensiva asimmetrica impostata dagli sviluppatori. Mentre il campo del messaggio subisce una sanificazione robusta, il campo relativo al nome dell'utente è protetto tramite un approccio basato su blacklist elementare.

```
<?php
if( isset( $_POST[ 'btnSign' ] ) ) {
    // Ricezione degli input dal modulo web tramite metodo POST
    $message = trim( $_POST[ 'mtxMessage' ] );
    $name     = trim( $_POST[ 'txtName' ] );

    // Sanificazione efficace del campo Message (Previene in modo assoluto XSS)
    $message = strip_tags( addslashes( $message ) );
    $message = htmlspecialchars( $message );

    // Sanificazione fallimentare del campo Name (Rilevamento del difetto logico)
    $name = str_replace( '<script>', '', $name );
```

```
// Esecuzione e archiviazione nel database della query SQL
$query = "INSERT INTO guestbook ( comment, name ) VALUES ( '$message',
'$name' );";
$result = mysqli_query($GLOBALS["__mysqli_ston"], $query);
}
?>
```

L'analisi puntuale evidenzia l'inefficienza intrinseca della funzione `str_replace()` applicata alla variabile `$name`. Tale funzione agisce rimuovendo esclusivamente la stringa letterale `'<script>'` in modalità case-sensitive e non ricorsiva. Questo modello difettoso espone il server all'evasione strutturale, in quanto non blocca l'utilizzo di tag HTML alternativi provvisti di event handler JavaScript integrati (come `onload` o `onerror`), idonei a forzare l'esecuzione di istruzioni arbitrarie sul client.

3. Evoluzione Operativa dell'Attacco: Analisi delle Due Operazioni

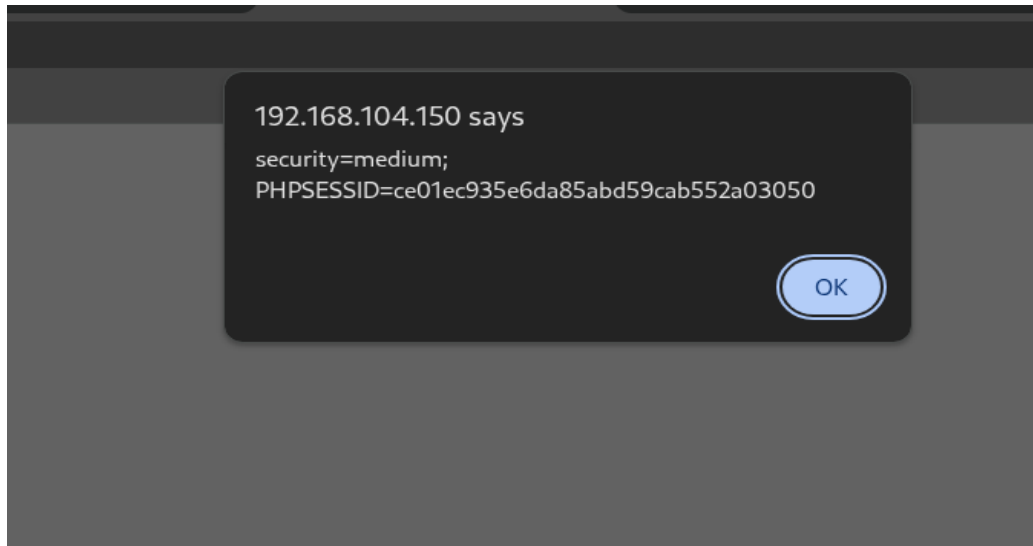
La replica dell'attacco ha seguito un percorso di ottimizzazione logica diviso in due distinte operazioni sequenziali, necessario per aggirare i limiti di parsing imposti dall'architettura dell'applicazione web e dalla serializzazione dei dati nel database.

Operazione 1: Verifica di Connessione ed Estrazione dei Soli Cookie

Inizialmente si è proceduto alla verifica della vulnerabilità iniettando un payload focalizzato esclusivamente sul recupero dei cookie di sessione della vittima. L'iniezione ha sfruttato un tag immagine fittizio programmato per generare un errore sistematico d'esecuzione ed attivare il modulo di alert nativo:

```
<img src=x onerror=alert(document.cookie)>
```

The image shows a screenshot of the DVWA (Damn Vulnerable Web Application) interface. On the left, a sidebar menu lists various security modules, with 'XSS stored' highlighted in green. The main content area displays the 'Vulnerability: Stored Cross Site Scripting (XSS)' section. It includes a form with 'Name' and 'Message' fields, and a 'Sign Guestbook' button. Below the form, a table shows previous entries, including one with the message 'W Hacker Empire'. On the right, a browser's developer tools are open, showing the HTML structure of the page. The 'Elements' panel highlights the 'div.vulnerable_code_area' which contains the guestbook form. The 'Console' panel shows the execution of the injected payload, resulting in an alert box displaying the document's cookies.

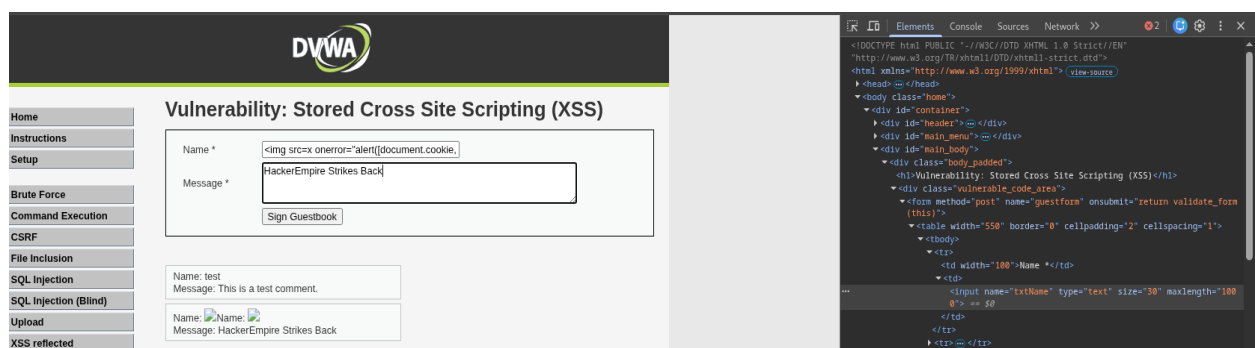


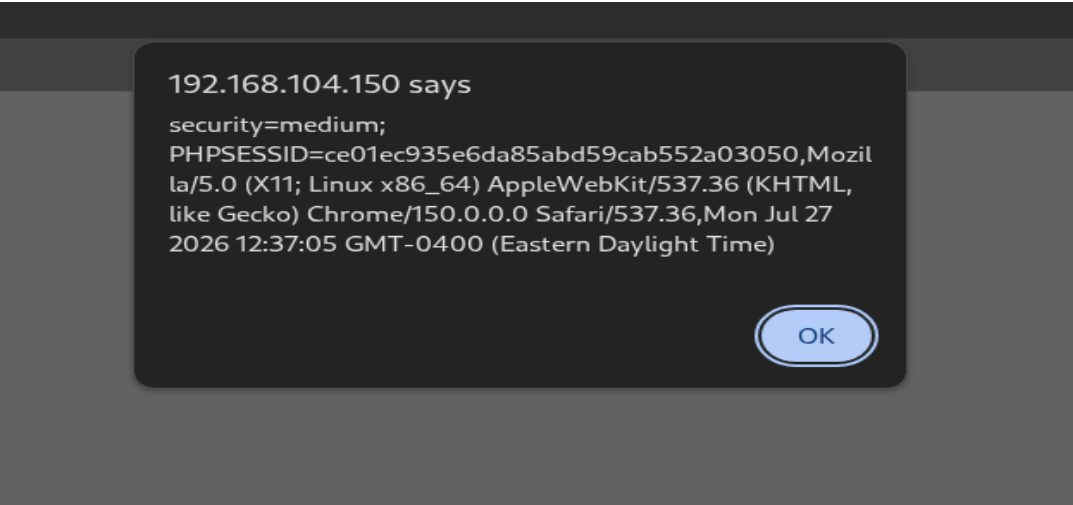
L'Operazione 1 ha confermato l'esistenza della falla, ma l'esigenza di estrarre un dump informativo organico (estendendo la raccolta alla tipologia di browser e alla marcatura temporale) ha richiesto una seconda fase di sviluppo. I tentativi intermedi volti a concatenare le stringhe tramite gli operatori convenzionali (segno + o sequenze di ritorno a capo \n) hanno determinato l'arresto anomalo dell'interprete JavaScript, poichè il database di DVWA interpretava letteralmente tali caratteri troncando la riga di codice in fase di rendering HTML.

Operazione 2: Sviluppo del Payload ad Array Nativo per il Dump Completo

Per superare in maniera definitiva i conflitti causati dai caratteri speciali di concatenazione e dagli apici nidificati, è stato ingegnerizzato un payload lineare ottimizzato basato su una struttura ad array nativo JavaScript ([...]):

```
<img src=x onerror="alert([document.cookie,navigator.userAgent,new Date])">
```





Questa intuizione architetturale risolve radicalmente le anomalie di memorizzazione poichè la sintassi ad array non richiede l'impiego di stringhe racchiuse tra apici protettivi esterni. Al momento dell'attivazione dell'attributo onerror, il motore JavaScript del browser interpreta gli oggetti in modo nativo, converte l'array in testo lineare e provvede ad unire automaticamente le tre proprietà richieste separandole con una virgola standard.

Mappatura Tecnica dei Parametri Estratti

Proprietà JavaScript	Informazione Estratta dal Client	Impatto Logico / Sicurezza
document.cookie	Token e identificativi alfanumerici della sessione attiva salvati nel client.	Consente l'acquisizione della sessione (Session Hijacking).
navigator.userAgent	Stringa descrittiva completa del software di navigazione e del sistema operativo.	Permette la mappatura delle vulnerabilità del client.
new Date	Data e ora correnti estratte dall'orologio interno del computer client della vittima.	Fornisce la tracciabilità temporale dell'esfiltrazione.

4. Guida Operativa Passo-Passo per la Replica dell'Attacco

Di seguito vengono documentati i passaggi esatti eseguiti nel laboratorio per condurre i test in modo controllato e privo di interferenze.

1. **Fase 1: Reset e Configurazione Ambiente:** Al fine di prevenire conflitti con i frammenti di codice interrotti e memorizzati nei test precedenti, accedere al menu 'Setup / Reset DB' di DVWA e fare clic su 'Create / Reset Database'. Successivamente, impostare il livello di sicurezza globale su 'Medium' nella scheda 'DVWA Security' e spostarsi sulla pagina 'XSS (Stored)'.
2. **Fase 2: Manipolazione del DOM Client-Side:** Il campo di inserimento 'Name' presenta una restrizione HTML locale che impedisce l'immissione di stringhe superiori a 10 caratteri. Posizionarsi sopra il campo, fare clic destro, selezionare la voce 'Ispeziona' e individuare nel DOM la riga contenente l'attributo `maxlength="10"`. Modificare il valore numerico a "500" e premere Invio per abilitare la scrittura dei payload complessi.
3. **Fase 3: Esecuzione dell'Operazione 1 (Verifica Cookie):** Incollare il codice `<img src=x onerror=alert(document.cookie)>` all'interno del campo Name sbloccato, inserire un testo di test nel campo Message e premere 'Sign Guestbook'. Al ricaricamento, confermare la visualizzazione del pop-up contenente unicamente il codice PHPSESSID.
4. **Fase 4: Esecuzione dell'Operazione 2 (Dump Completo):** Dopo aver eseguito nuovamente il reset del database per ripulire la pagina (come descritto nella Fase 1), sbloccare nuovamente il parametro `maxlength`. Incollare nel campo Name il payload avanzato ad array nativo: `<img src=x onerror="alert([document.cookie,navigator.userAgent,new Date])">`. Compilare il campo Message con la stringa 'Test Concluso' e premere 'Sign Guestbook'. Il sistema elaborerà l'input mostrando l'alert finale unificato comprensivo di Cookie, stringa User-Agent e Data/Ora locale.

5. Strategie di Mitigazione e Remediation

Il superamento dei controlli basati su blacklist dimostra che la sicurezza di un'applicazione non può fondarsi sull'intercettazione parziale di parole chiave note. Per sanare la vulnerabilità del modulo, è necessario sostituire la logica di filtraggio del file PHP centralizzando la conversione sintattica dei caratteri speciali.

La riga di backend difettosa:

```
$name = str_replace( '<script>', '', $name );
```

Deve essere obbligatoriamente sostituita con la funzione di codifica nativa di PHP:

```
$name = htmlspecialchars( $name, ENT_QUOTES, 'UTF-8' );
```

L'adozione della funzione `htmlspecialchars()` impone al server di convertire i caratteri di controllo strutturali dell'HTML in entità testuali inerti (ad esempio il simbolo minore viene codificato come `<`). Il flag `ENT_QUOTES` estende il trattamento sia ai singoli che ai doppi apici, neutralizzando ogni tentativo di evasione dai tag o dagli attributi. Quando il client riceve i dati così manipolati, il browser esegue il rendering grafico dei simboli a schermo interpretandoli come testo statico, azzerando istantaneamente il rischio di esecuzione JavaScript arbitraria.